SITECONTROL

Administrator Guide

Installing posts, creating and retiring sites, configuration, passwords and keys,
backups, restores, upgrades and health checks.

For IT / administrators - not for gate officers. Written against the deployed sitectl build of 2026-10-08.

Contents

Section	Covers	Page
About This Guide	Who it is for, how to read the TESTED marks	3
How It Fits Together	Posts, the mesh, the front door	4
The sitectl Command	Service flags and the admin command set	5
site.toml Reference	Every setting and what it accepts	6
Installing a Post	Linux / systemd install, first start	8
Creating New Sites	A whole new season, or one more post	10
Retiring a Site	Removing a post from the mesh	12
Passwords and Keys	Changing the site password, peer keys	13
Backups and Restore	Nightly copies, manual copies, restoring	14
Replacing or Upgrading	Laptop swap, new binary	16
Season End and Start	Shutting down and coming back up	17
Health Checks	Is it up, is it in sync, what the logs mean	18
Windows Install	Running a post as a Windows service	19
Known Gaps	What is not yet documented or tested	20
Admin Quick Reference	One-page summary	21

About This Guide

This guide is for whoever installs and looks after SiteControl: setting up the computer at each post, creating a new site, taking an old one out of service, changing passwords, and keeping backups. Gate officers use the **Site Operator Guide**; dispatch uses the **Dispatch Guide**.

TESTED marks

Every procedure carries one of these:

TESTED *the steps were run end to end on a test copy of SiteControl and worked as written.*

NOT YET TESTED *the steps come from the program's own help text and the live installation, but have not been run start to finish. Do them carefully the first time and confirm each result.*

Secrets. Each post's **site.toml** holds its login password and every peer key. Never email it, print it, or paste it into a chat. In this guide secrets are shown as placeholders: **<SITE_PASSWORD>**, **<PEER_KEY>**, **<PROXY_TOKEN>**.

To look at a site.toml safely, show the key names only, or mask the secret lines:

```
grep -o '^ *[A-Za-z_.]*' site.toml
sed -E 's/^( *(peer_key|key|login_password|proxy_auth_token) *=).*/\1 <masked>/' site.toml
```

How It Fits Together

1	**One program per post.** Each gate (post) runs its own copy of **sitectl** with its own **site.toml** and its own SQLite database. A post works fully on its own - no internet needed to record anything.
2	**The mesh.** Every post lists every other post as a **peer** and pulls their records every few seconds (**sync_interval_seconds**, normally 10). Each pair shares a key. When the link drops, posts carry on and catch up later.
3	**Records are never overwritten.** Every check-in, departure and correction is an event added to a log. Corrections add to the record; nothing is erased.
4	**The front door** (only where posts are reached through a browser from outside): a web proxy (Caddy) per host, then a Cloudflare tunnel and a hostname per post.

The current demo installation

Host	A single Linux container on the demo host
Program	/opt/sitectl/sitectl - older copies kept beside it as sitectl.pre-YYYYMMDD-HHMM
Posts	One folder each under /opt/sitectl/<name>/ - iceroad :8081, pad :8082, npf :8083, ndb :8084, sid :8085 (all 127.0.0.1)
Note	The folder / service name is not always the site_id: folder **pad** runs site_id **ice-pad**; **iceroad** runs **ice-road**.
Service	systemd template **sitectl@.service** - one instance per folder (sitectl@pad, ...)
Front door	Caddy on :80 (/etc/caddy/Caddyfile) -> Cloudflare tunnel -> sc-<name>.jnkfacey.com; landing page sc-demo.jnkfacey.com
Helpers	/root/sc-converge.py (wait for all posts to agree), /root/demoseed/ (demo data)

The service unit

```
[Service]
User=sitectl
WorkingDirectory=/opt/sitectl/%i
ExecStart=/opt/sitectl/sitectl -config /opt/sitectl/%i/site.toml
Restart=on-failure
RestartSec=2
[Unit]  After=network-online.target  Wants=network-online.target
[Install]  WantedBy=multi-user.target
```

The sitectl Command

TESTED *every flag below was run.*

Service and maintenance flags

Flag	What it does
-config FILE	The post's site.toml. Required for everything. On its own: run the post (what systemd does).
-backup DIR	Make one verified backup copy into DIR and exit. Safe while running. Keeps the 7 newest in DIR.
-restore FILE	Replace this post's database with a backup and exit. Refuses while the post is running.
-dump	Print the post's full state as JSON - compare two posts to prove they agree.
-report visits	CSV of every visit and time on site (read-only).
-report signout-failures	CSV of missed sign-outs (read-only).
-h	Help. There is **no -version** flag - identify a build by file size / sha256sum.

Admin commands (run by IT, never by the service)

```
sitectl admin season      -plan site-plan.toml -out DIR       # build a whole new mesh
sitectl admin check       a/site.toml b/site.toml ...           # prove config files agree
sitectl admin site add    -config LIVE.toml -self-url URL -new new-post.toml -out DIR
sitectl admin peer apply  -config site.toml join-<id>.toml      # at each existing post
sitectl admin site retire -config site.toml -out DIR            # at the post leaving
sitectl admin peer remove -config site.toml leave-<id>.toml     # at each remaining post
sitectl admin swap plan   -config site.toml [-old-url URL] [-from-backup FILE]
sitectl admin swap status -config site.toml [-wait 30m]         # READY check
```

Common admin options: **-db FILE** (the database, if it is not beside site.toml) and **-out DIR** (must be new or empty). The admin tools never print passwords or keys; keys are named by fingerprint (fp:xxxxxxxx).

Gotcha (tested): the running service finds a relative **db_path** from its working folder; the admin tools look beside site.toml. They agree under systemd (the working folder is the post's folder). If you ever run the program by hand from elsewhere, pass **-db** to the admin tools or use an absolute db_path.

site.toml Reference

The loader is **strict**: an unknown key or a wrong type stops the post from starting, and the error names the key. Top-level settings must come **before** the first [[table]].

TESTED *each rule was produced by running the program against a bad file.*

Setting	Meaning / allowed values
site_id	Permanent short name, e.g. "ice-pad". Never reuse a retired id.
display_name	Name shown on every page.
listen_addr	"host:port" - e.g. "127.0.0.1:8082" behind a proxy. A port on its own is refused.
db_path	The SQLite file. Use an absolute path on Windows.
node_epoch	Whole number, 1 or more. Set at provisioning; raised by swap plan after a restore or machine swap. Never lower it.
peer_key	The key this post accepts from peers calling it (64 hex characters).
login_password	The one site password officers type (required). No usernames.
login_hint	Optional text on the login page.
proxy_auth_token	Shared secret the proxy sends in the X-SC-Proxy-Auth header. Only allowed with a loopback listen_addr.
dev_routes	true/false. **Must be false** on anything reachable from outside.
orientation	true at a work site that gives its own orientation; false at a road post.
site_type	"camp" (people live there; returning without signing out is normal) or "text_message" (per-visit sign in/out).
muster_min_group_size	Crews smaller than this are pooled into the muster's Summary group (generated default 5).
sync_interval_seconds	How often to pull from peers (generated 10). A peer is STALE / LOST SIGNAL after 3x this.
shift_starts	e.g. ["06:00","18:00"] - the gate's shift counters reset at these times. Unset = 06:00 and 18:00.
theme_cookie_domain	Optional, e.g. ".jnkfacey.com", so light/dark mode follows across posts.
backup_dir	Folder for nightly copies. Unset = "backups".
backup_mirror_dir	Optional second folder (another disk / USB) - every nightly copy is written to both.
backup_time	"HH:MM" local time for the nightly copy. Unset = about 03:00 (each post runs a few minutes apart).

Tables

```
[[peers]]                      # one per OTHER post
site_id      = "ice-road"
display_name = "2P Ice Road Entrance"
long_name    = "..."            # optional
base_url     = "http://127.0.0.1:8081"   # how THIS post reaches that post
key          = "<PEER_KEY>"     # that post's peer_key

[[destinations]]               # where this post may dispatch vehicles
id             = "ice-road"     # a peer's site_id
display_name   = "2P Ice Road Entrance"
controlled     = true           # true = a gate at that end, the run is timed
travel_minutes = 65             # one number, same both ways; prefills the deadline

[[former_peers]]               # written by 'admin peer remove' for a retired post
site_id      = "gamma"
display_name = "Gamma Gate"
```

Mesh rule	For every pair A and B: A's [[peers]] entry for B carries B's peer_key, and the other way round; every base_url agrees. **admin check** proves this.
destinations	With two or more peers the list must be written out explicitly (destinations = [] if none).
Removed	Weather phases are gone: a [[weather.phase]] table or a per-phase travel_minutes table is refused. travel_minutes is one number.
Not yet documented	entrance_site_id, one_way, road_work, road_work_label, limit_minutes, ungated - present in the program, meaning not confirmed. Leave them as generated.

Installing a Post

NOT YET TESTED *the program and its config were tested; the systemd and proxy steps follow the live installation.*

1	**Once per host** - install the program, a service user and the unit:
2	**Get the post's site.toml** - from Creating New Sites (a new season or one more post).
3	**Put it in place** (owner sitectl, mode 0640 - the file holds secrets):
4	**Start it:** systemctl enable --now sitectl@<name>
5	**Check it** (next page).

```
# step 1
useradd --system --home /opt/sitectl sitectl
install -o sitectl -g sitectl -m 0755 sitectl /opt/sitectl/sitectl
cp sitectl@.service /etc/systemd/system/ && systemctl daemon-reload

# step 3
install -d -o sitectl -g sitectl -m 0750 /opt/sitectl/<name>
install -o sitectl -g sitectl -m 0640 site.toml /opt/sitectl/<name>/site.toml

# step 4
systemctl enable --now sitectl@<name>
```

Front door (only if the post is reached from outside)

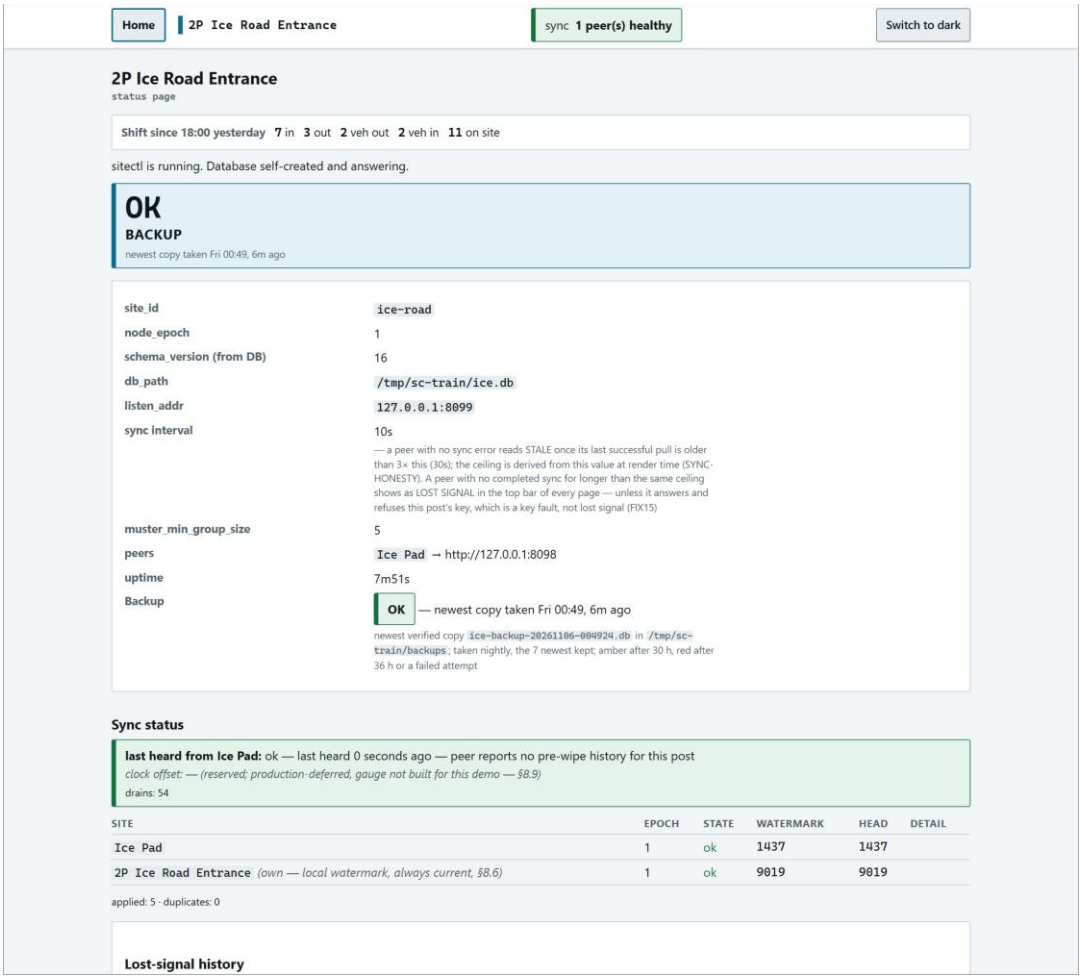
```
http://sc-<name>.jnkfacey.com {
    import common
    import gate
    reverse_proxy 127.0.0.1:<port> {
        header_up X-SC-Proxy-Auth "<PROXY_TOKEN>"     # must equal the post's
proxy_auth_token
    }
}
# then:
caddy validate --config /etc/caddy/Caddyfile && systemctl reload caddy
```

Then add **sc-<name>.jnkfacey.com** to the Cloudflare tunnel (pointing at the Caddy host on port 80) with its DNS record. The tunnel's hostnames are managed in the Cloudflare dashboard, not in a config file - see Known Gaps.

First start - what you should see

TESTED

Journal	journalctl -u sitectl@<name> -n 20 shows "site=<id> epoch=1 ... listening on <addr> (sync every 10s, N peer(s))" and the next backup time.
Peers not up yet	"REJOIN ... FAILED" and "PEER UNREACHABLE" are normal while posts start one by one; within 10-20 s "PEER REACHABLE AGAIN" / "SYNC RECOVERED".
Database	Created automatically on first start.
/status page	No login needed: "sitectl is running. Database self-created and answering.", backup state, site id, epoch, peers and sync state.
Login	One field, Site password, then Individual / Vehicle check-in. The page then reads "Recording as: <display_name>".
First backup	About 2 minutes after the first start, then nightly.



The status page of a running post.

Creating New Sites

A. A whole new set of posts (new season)

TESTED *two posts.*

1	Write a plan file listing every post and every road between them:
2	Run **admin season** into a new, empty folder.
3	It writes one **<site_id>/site.toml** per post, **passwords.txt** (the officer handover sheet, one password per post) and **IT-CHECKLIST.txt** (no secrets). If the plan is inconsistent it prints **REFUSED - nothing was written** and the reasons.
4	Optionally add backup_dir, backup_time, backup_mirror_dir, proxy_auth_token, theme_cookie_domain or shift_starts to each file (top of the file). Not generated for you.
5	Install each post (Installing a Post), start them all, then prove the mesh (below). **Delete the output folder** - it is the only other copy of every key.

```
# site-plan.toml
[[post]]
site_id      = "alpha"
display_name = "Alpha Gate"
listen_addr  = "127.0.0.1:8090"
base_url     = "http://127.0.0.1:8090"
site_type    = "camp"            # or "text_message"
orientation  = true              # required: true at a work site, false at a road post
[[post]]
site_id      = "beta"
...
[[route]]
a = "alpha"
b = "beta"
travel_minutes = 60


# run it
sitectl admin season -plan site-plan.toml -out /root/season-out
```

Expected: one line per post ("accepts fp:..., 1 peer(s), 1 destination(s)"), then "cross-check: every post lists every other, every key held matches the key accepted, every base_url agrees".

Prove the mesh

```
sitectl admin check alpha/site.toml beta/site.toml ...
#   MATCH: every post lists every other, every key held is the key accepted, every address
agrees
sitectl admin swap status -config alpha/site.toml
#   READY: alpha has its own history back and is in step with every peer   (exit code 0)
```

B. Add one post to a running set

TESTED *a third post added to a running pair.*

1	Write **new-post.toml** with exactly ONE [[post]] (the newcomer) and its [[route]] entries.
2	On any one existing post (it may stay running), run **admin site add**. It writes the newcomer's site.toml, one **join-<id>.toml** (secret - carries the newcomer's key) and **passwords-<id>.txt**.
3	At **each** existing post run **admin peer apply** with that same join file, then restart that post. The old file is kept as site.toml.bak-<date>.
4	Install and start the new post. Check: **swap status** on the new post says READY; /status on every post shows the newcomer in sync.
5	Add the new post to the proxy and tunnel, and make sure the other posts can reach its base_url (firewall).

```
sitectl admin site add -config /opt/sitectl/<existing>/site.toml \
      -self-url http://<how the other posts reach this existing post> \
      -new new-post.toml -out /root/add-out

# at EACH existing post:
sitectl admin peer apply -config /opt/sitectl/<existing>/site.toml join-gamma.toml
systemctl restart sitectl@<existing>
```

Expected at peer apply: "added peer gamma ...", "added destination gamma (gated, 45 min)" at the post named in the route. Running it twice is refused ("gamma is already a peer here - nothing to do").

Retiring a Site

TESTED *on a post with an empty database - the "someone still on site" refusal was not triggered.*

1	Make sure **nobody is on site** and **no vehicle run is open** at that post. The tool checks and refuses otherwise.
2	**Stop** the retiring post: systemctl stop sitectl@<name>. (While it runs the tool refuses: "service still answers ... stop the service, then run this again".)
3	With the other posts running, run **admin site retire**. It proves every other post already holds all of this post's records, then writes **leave-<id>.toml** (no secrets).
4	At **each remaining** post run **admin peer remove** with that file, then restart it. The retired post becomes a [[former_peers]] entry so its name stays on old records.
5	Run **admin check** on the remaining files - MATCH.
6	Take and keep a **final backup** of the retired post's database. Remove its proxy block, tunnel hostname, DNS record and firewall entries. Wipe the machine.

```
sitectl admin site retire -config /opt/sitectl/<name>/site.toml -out /root/leave-out
#   ... holds every record of Gamma Gate
#   nobody on site, no open run, every peer holds Gamma Gate's records: Gamma Gate may retire

# at EACH remaining post:
sitectl admin peer remove -config /opt/sitectl/<remaining>/site.toml leave-gamma.toml
#   removed peer gamma / removed 1 destination(s) to gamma / kept its name: [[former_peers]]
systemctl restart sitectl@<remaining>
```

A retired site_id can never come back. Its records stay in every remaining post's history. A post that returns later must use a **new site_id** (added with Creating New Sites, B).

Passwords and Keys

Site login password

NOT YET TESTED

1	Edit **login_password** in the post's site.toml (keep a copy of the old file first: cp site.toml site.toml.bak-<date>).
2	Restart: systemctl restart sitectl@<name>.
3	Tell the officers at that post. There is no user list and no web page for passwords - one password per post.

To make one in the same style the tools use (xxxx-xxxx-xxxx-xxxx):

```
openssl rand -hex 8 | sed 's/..../&-/g;s/-$//'
```

Peer keys

NOT YET TESTED *there is no rekey command.*

A peer key must be changed at **both ends at once**: the accepting post's **peer_key** and the matching **key** in the other post's [[peers]] entry. Restart both, then run **admin check** - MATCH. A mismatch shows on /status as a key fault (the peer "refuses this post's key"), which is different from LOST SIGNAL.

Other settings

backup_time, shift_starts, destinations and so on: edit, restart, and read journalctl -u sitectl@<name> - every config error names the setting. Always keep the previous file.

Backups and Restore

Nightly backups - automatic

TESTED

When	Every night at backup_time (about 03:00; posts are a few minutes apart). First copy ~2 minutes after a fresh start.
How	Online SQLite copy, then verified (integrity check and a hash check of every event). Log: "backup: verified copy <path>".
Kept	The 7 newest copies, named <site_id>-backup-YYYYMMDD-HHMMSS.db, in backup_dir (default "backups").
Second copy	Set backup_mirror_dir to another disk or USB - every copy is written to both.
Check	/status shows "Backup OK - newest copy taken ..., N ago". Investigate if older than a day.

A copy in the same folder as the database dies with that disk - use backup_mirror_dir, and keep copies off the machine.

Manual backup (any time, running or not)

```
sitectl -config /opt/sitectl/<name>/site.toml -backup /root/manual-bak
```

Restoring a post's database

TESTED *mechanics tested on an empty database.*

1	Stop the post: systemctl stop sitectl@<name>.
2	Pick the copy from backups/. If the post has been used at all, restore with **swap plan -from-backup** (it raises node_epoch, which a plain -restore requires).
3	The database being replaced is kept as <db>.pre-restore-<date>.
4	Start the post. On start it pulls its own newer records back from its peers before serving.
5	Wait for READY: sitectl admin swap status -config site.toml -wait 30m.

```
sitectl admin swap plan -config /opt/sitectl/<name>/site.toml -from-backup backups/<copy>.db
#   node_epoch 1 -> 2 (old file kept as site.toml.bak-...)
#   next: start the SiteControl service, then run sitectl admin swap status ... until it says
READY
systemctl start sitectl@<name>
sitectl admin swap status -config /opt/sitectl/<name>/site.toml -wait 30m

# plain restore - only on a post that has never been used in its current epoch:
sitectl -config /opt/sitectl/<name>/site.toml -restore backups/<copy>.db
```

swap plan asks the other posts for their highest record of this post; if one does not answer it refuses. Only if that post is truly gone: add **-epoch N -unreachable-ok** with N above any epoch it could hold.

Replacing or Upgrading

Replacing a post's machine (laptop swap)

NOT YET TESTED

1	Switch the old machine off or unplug it. (swap plan refuses while the old one still answers.)
2	On the replacement, with the post's site.toml copied over and **before the first start**, run swap plan (optionally -from-backup with the newest copy).
3	Start the service and wait for READY with swap status -wait 30m.

```
sitectl admin swap plan -config site.toml -old-url http://<old machine>:<port> [-from-backup <copy>]
```

Upgrading the program

NOT YET TESTED *follows how the demo has been upgraded.*

1	Back up every post first (manual -backup, or copy the post folders).
2	Keep the old program: cp -a /opt/sitectl/sitectl /opt/sitectl/sitectl.pre-$(date +%Y%m%d-%H%M)
3	Copy the new program in as sitectl.new, then mv it over sitectl.
4	Restart each post: systemctl restart sitectl@<name>. Check /status and the journal.

No going back after a database upgrade. A newer program may upgrade the database; an older program then refuses it ("schema_version N but this binary requires M - refusing to open"). To roll back, restore the pre-upgrade backup with the old program. Run the **same build at every post** - compare with sha256sum /opt/sitectl/sitectl.

Season End and Start

NOT YET TESTED *assembled from the tools; there is no written season-end procedure yet.*

Season end

1	Take a final backup from every post (-backup) and store it off the machines.
2	Export what the business needs: -report visits and -report signout-failures (CSV); optionally -dump as an audit copy.
3	Shut the posts down: systemctl stop sitectl@<name>.

Next season - two choices

Keep the data	Power the posts up with their partners running. The journal shows REJOIN then SYNC RECOVERED; swap status says READY. Check the clock is right first - a clock set back is flagged on the records.
Start clean	Run admin season with a new plan (Creating New Sites, A). New databases start at node_epoch 1. Do not mix new posts with old databases of the same site_id.

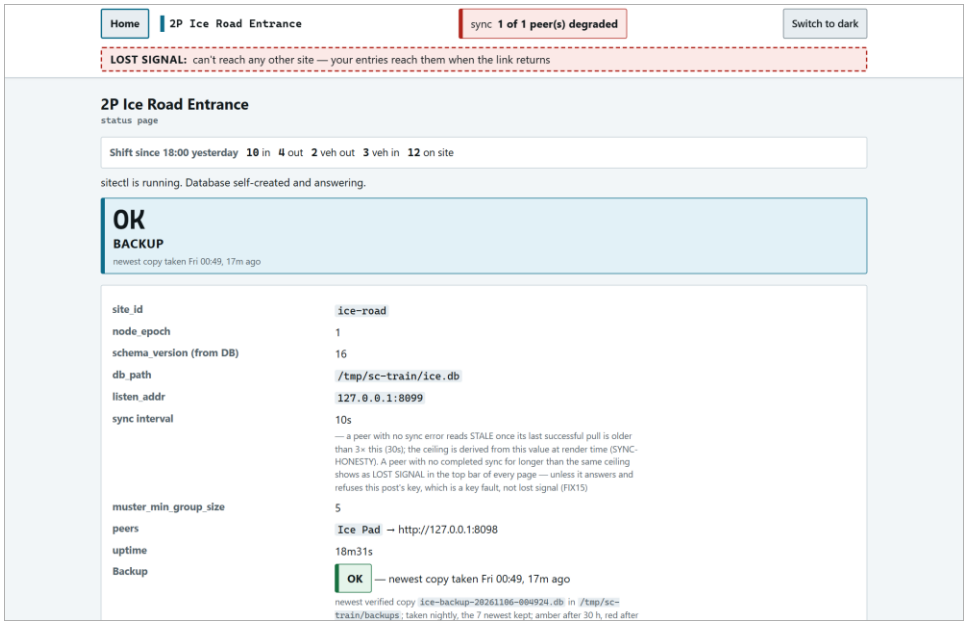
Machines that sit unpowered for long periods: see the hardware report for battery, storage and condensation advice before powering up in the cold.

Health Checks

Check	Command	Healthy
Running	systemctl is-active sitectl@<name>	active
Answering	curl -s -o /dev/null -w '%{http_code}' http://127.0.0.1:<port>/status	200
Sync detail	curl -s http://127.0.0.1:<port>/debug/sync-status	every stream state ok; no key refused; nothing pending
Configs agree	sitectl admin check <all site.toml>	MATCH
Caught up	sitectl admin swap status -config site.toml	READY (exit 0)
Posts agree	sitectl -config site.toml -dump (on two posts, compare)	identical
Backup fresh	/status - Backup row	newest copy under 24 h old

Log messages

Message	Meaning
PEER UNREACHABLE	The other post is down, firewalled, or its base_url is wrong.
REJOIN ... FAILED	Normal at start-up while peers come up. If it persists, a peer is unreachable.
PEER REACHABLE AGAIN / SYNC RECOVERED	Good - the link is back.
schema_version N but this binary requires M	Program and database versions do not match - see Upgrading.
sitectl: config FILE: ...	A setting is wrong; the post will not start (systemd retries every 2 s). The message names the setting.
bind: address already in use	Another program holds the port.
backup: SECOND COPY FAILED	backup_mirror_dir is not writable. The main copy is fine.



/status while a post has lost contact with its peers.

Windows Install

NOT YET TESTED *from the September installer script; the October installer was not available.*

1	Run the installer (**deploy\windows\install-service.ps1**) from an elevated PowerShell. It copies sitectl.exe to C:\Program Files\SiteControl\, creates C:\ProgramData\SiteControl\ (Administrators + the service only), and registers the **SiteControl** service (virtual account, automatic start, restart on failure). Re-running it replaces the program or config.
2	Put the post's config in C:\ProgramData\SiteControl\. **Check the file name the service expects**: the September script uses **config.toml**, the newer checklist says **site.toml** - match the service's -config path.
3	Set **db_path** to an absolute path, e.g. C:\ProgramData\SiteControl\site.db (a service has no fixed working folder).
4	Firewall: allow the post's listen port inbound **only** from the other posts' addresses (listed in IT-CHECKLIST.txt).

Uninstall with deploy\windows\uninstall-service.ps1.

Known Gaps

Things this guide could not yet confirm. Check them before relying on them in production:

Cloudflare hostnames	How sc-<name>.jnkfacey.com is added to the tunnel - managed in the Cloudflare dashboard, not written up.
Some settings	entrance_site_id, one_way, road_work, road_work_label, limit_minutes, ungated, login_hint: present in the program, meaning not confirmed.
Rekey	No command to change a peer key; manual both-ends edit, untested.
Windows	The current installer and its config file name (config.toml vs site.toml).
Real sites	How posts reach each other across separate Starlink links in production (a private VPN link is expected; not designed yet).
Real data	Retire, restore and swap were tested on empty databases; the refusals for people on site / open runs were not triggered.
Passwords	Whether a minimum password length is enforced.
Source	The program's source repository is not on this laptop; this guide follows the deployed 2026-10-08 build.

Admin Quick Reference

New season	admin season -plan site-plan.toml -out DIR > install each > admin check > swap status
Add a post	admin site add (on one post) > peer apply + restart (each existing post) > install new post
Retire a post	nobody on site > stop it > admin site retire > peer remove + restart (each remaining) > admin check
Change password	edit login_password > restart > tell the officers
Manual backup	sitectl -config site.toml -backup DIR
Restore	stop > admin swap plan -from-backup COPY > start > swap status -wait 30m
Upgrade	backup > keep sitectl.pre-<date> > replace > restart each post
Is it healthy	/status 200 > /debug/sync-status ok > admin check MATCH
Logs	journalctl -u sitectl@<name> -n 50
Never	email or print site.toml; reuse a retired site_id; lower node_epoch